

PORTFOLIO

# 일정관리 앱 사용 가이드

## 감사업무관리 기능 중심 소개

일정관리 · 채용공고 자동수집 · 감사업무관리(트리/칸반) — Electron 데스크톱 앱

---

Electron · SQLite · Python · GitHub API/Actions 기반 자체 개발 애플리케이션

\* 개인 로컬 환경에서 개발되어 GitHub 저장소에는 민감정보(토큰/경로 등)를 제외하고 업로드되어 있습니다.  
본 문서는 실제 코드와 사용 화면 캡처를 기반으로 한 동작 확인용 사용 가이드입니다.

# 목차



01

## 개요

왜 만들었나(페인포인트) · 하나의 앱, 세 가지 기능



02

## 모바일 지원

감사업무관리 제외 전 기능 모바일 지원 + 구현 방식



03

## 감사업무관리 구조

무한 계층 트리(AuditNode) 아키텍처



04

## 실행 화면

트리 뷰 · 칸반 뷰 · 작업상세 · 엑셀 자동인식(실제 회계 현업 프로세스를 반영한 **모의 가상 데이터**를 활용하여 핵심 기능을 시연)



05

## 구현 방식

Python ↔ Electron 창 추적 파이프라인



06

## 채용공고 자동수집

KICPA 크롤링 자동화 구현 방식



07

## 그 외 기능

일정관리 (요약)



08

## 마무리

기술 스택 정리

## 왜 만들었나



### 문제의식

- KICPA 채용공고를 매번 사이트에 직접 들어가 확인해야 해서 새 공고나 마감을 놓치기 쉬움
- 감사 현장에 투입되면 여러 클라이언트의 조서 작업을 동시에 진행하게 되어, 지금 무엇을 어디까지 했는지 파악하기 어려움
- 개인 일정과 회사별 감사 업무가 따로따로 관리되어 마감을 놓치기 쉬움



### 해결 방식

- GitHub Actions로 채용공고를 자동 크롤링하고, 마감 하루 전 카카오톡으로 알림
- 감사 업무를 회사 → 조서 → 세부업무의 트리 구조로 관리하고, 지금 열려 있는 Excel 조서를 자동 인식해 관련 업무를 바로 보여줌
- 일정관리와 감사업무관리를 하나의 앱으로 통합, GitHub API로 데스크톱↔모바일 동기화

# 하나의 앱, 세 가지 기능

이 중 이번 가이드에서는 감사업무관리 기능을 중심으로 다룹니다



## 일정관리

모바일 지원

달력 기반 개인 일정 · 진행률 관리



## 채용광고 자동수집

모바일 지원

KICPA 채용광고 크롤링 + 마감 알림



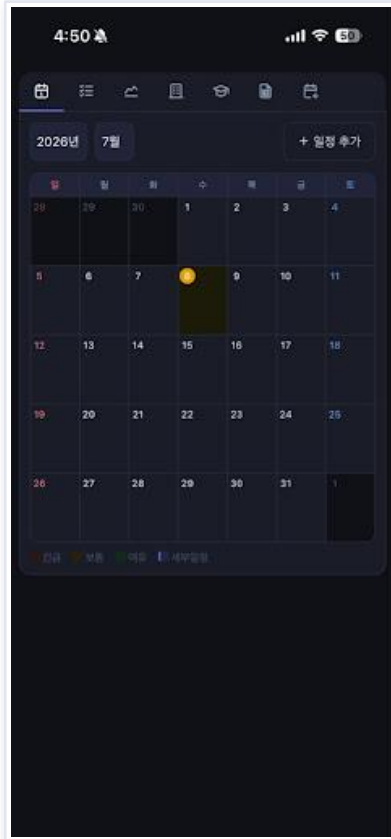
## 감사업무관리

이 가이드의 핵심

보안상 데스크톱 전용 — 모바일에서는 조회만 가능

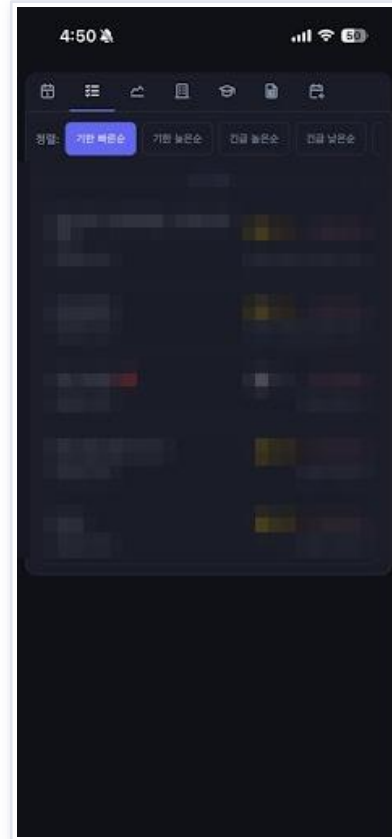
- 감사 현장에서 진행 중인 업무를 무한 계층 트리(부모-자식) 구조로 관리
- 트리 뷰 / 칸반 뷰 두 가지 방식으로 같은 데이터를 전환하며 확인
- 업무별 메모 · 체크리스트(할일) · 진행 상태를 SQLite에 저장
- 자료징구(요청 문서) 현황을 별도 화면에서 추적

# 모바일에서는 이렇게 보입니다



달력

 모바일 지원



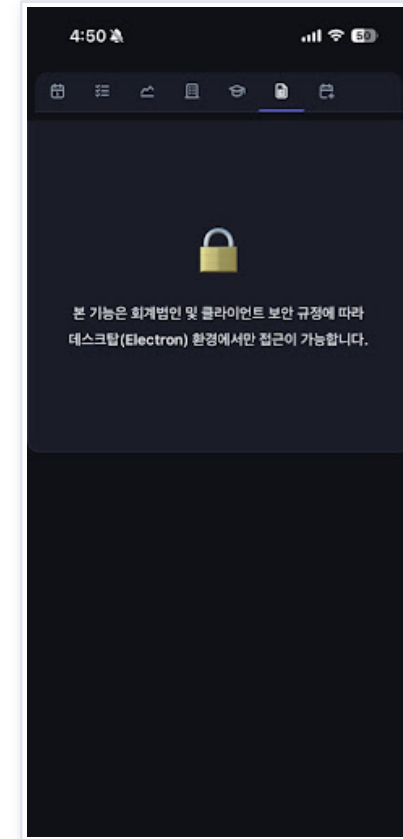
일정 목록

 모바일 지원



채용공고

 모바일 지원



감사 업무 (접근 제한)

 데스크톱 전용

## 모바일에서는 어떻게 되나: 구현 방식과 예외



### 모바일 · 웹 지원 방식

- Electron 데스크톱 앱과 별개로, 같은 UI를 Vercel에 웹으로 배포 — GitHub push 시 자동 재배포
- 브라우저(모바일)는 GitHub Personal Access Token을 직접 갖지 않음 — Vercel 서버리스 함수(api/\*.js)가 Vercel 환경변수에 저장된 토큰으로 GitHub API를 대신 호출하는 프록시 구조
- 일정 데이터(tasks.json)는 GET·PUT 프록시가 모두 구현되어 있어, 모바일에서도 읽기·쓰기가 자유롭고 데스크톱과 실시간 동기화됨



### 감사업무관리는 왜 예외인가

- Vercel 배포는 곧 코드와 데이터가 외부에 호스팅된다는 뜻 — 공개 배포 자체가 없던 공격 표면을 새로 만드는 것
- 감사 데이터(클라이언트명·조서 내용 등)는 특히 민감해, 쓰기 가능한 통로가 하나라도 생기면 리스크가 큼
- 그래서 audit\_snapshot.json 프록시는 GET(조회)만 구현하고 PUT(쓰기)은 아예 만들지 않음 — 화면에서도 추가·수정 버튼을 모바일에서 숨김(CSS)
- 결과: 모바일 웹이 뚫리더라도 감사 업무 데이터를 변조할 수 있는 통로 자체가 서버에 존재하지 않음

## 무한 계층 트리 구조 (AuditNode)



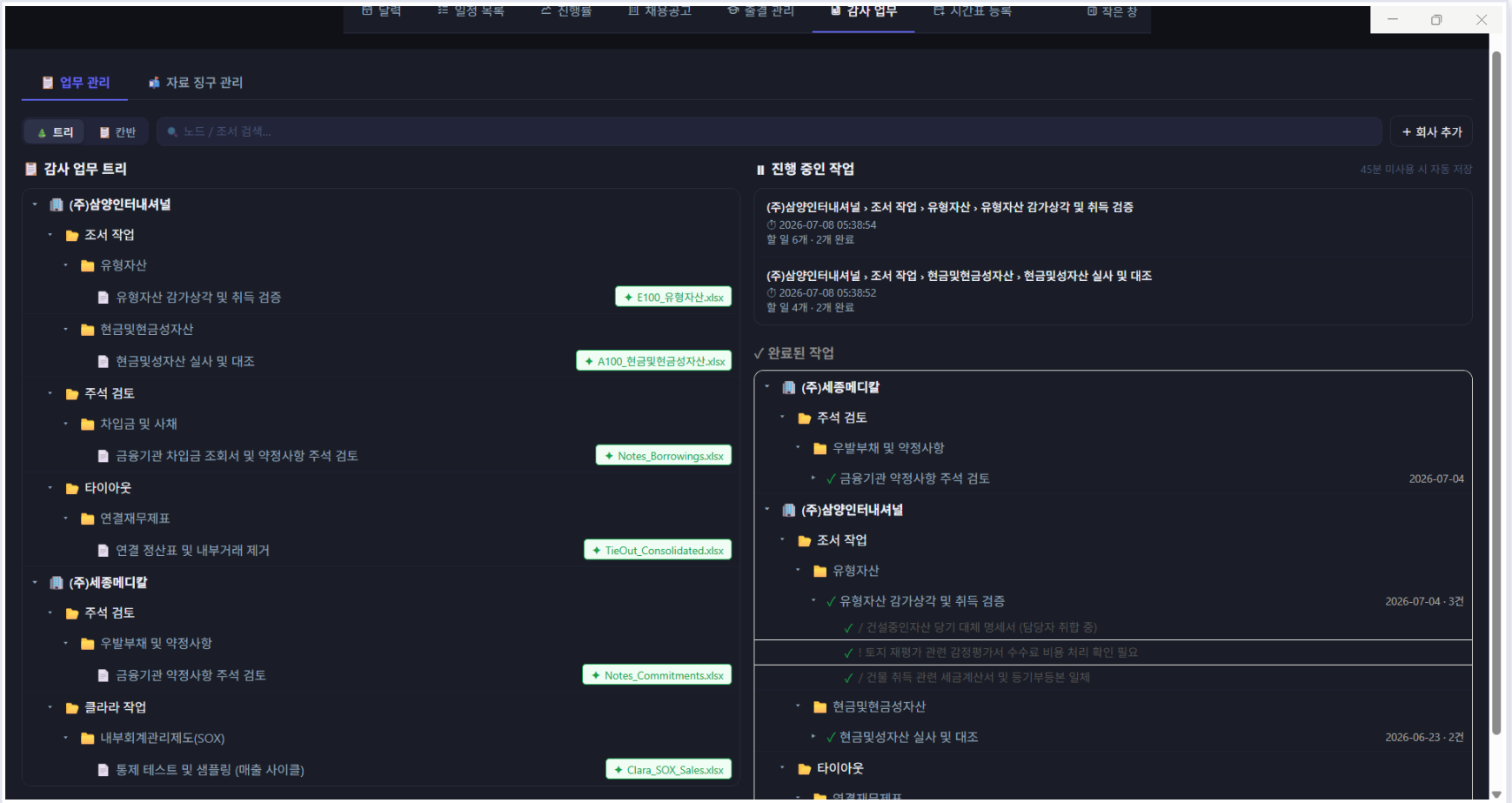
### 노드 하나가 담는 데이터

- 이름 (name)
- 메모 (최대 5,000자)
- 할일 체크리스트 (todos, JSON)
- 진행 상태
- 상위 업무 참조 (parent\_id)

better-sqlite3(SQLite)로 로컬 저장, 입력값 길이 제한 등 가드 로직 적용

\* 실제 예시 데이터가 아닌 구조 설명용 가상의 이름입니다. parent\_id 자기참조로 깊이 제한 없이 하위 업무 추가 가능

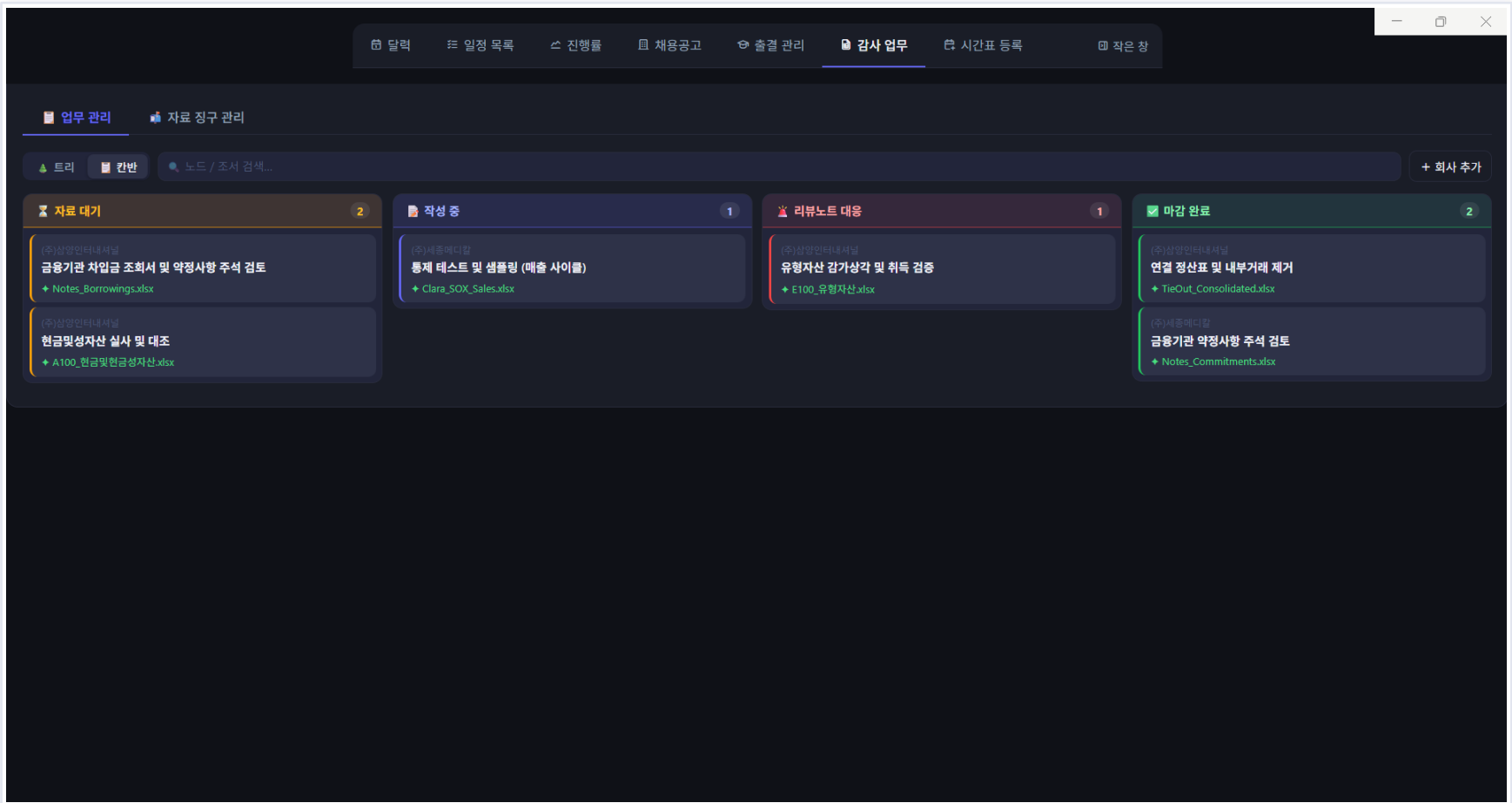
# 실행 화면 ① 트리 뷰



계층형 트리로 상위-하위 업무 관계를 한눈에 파악

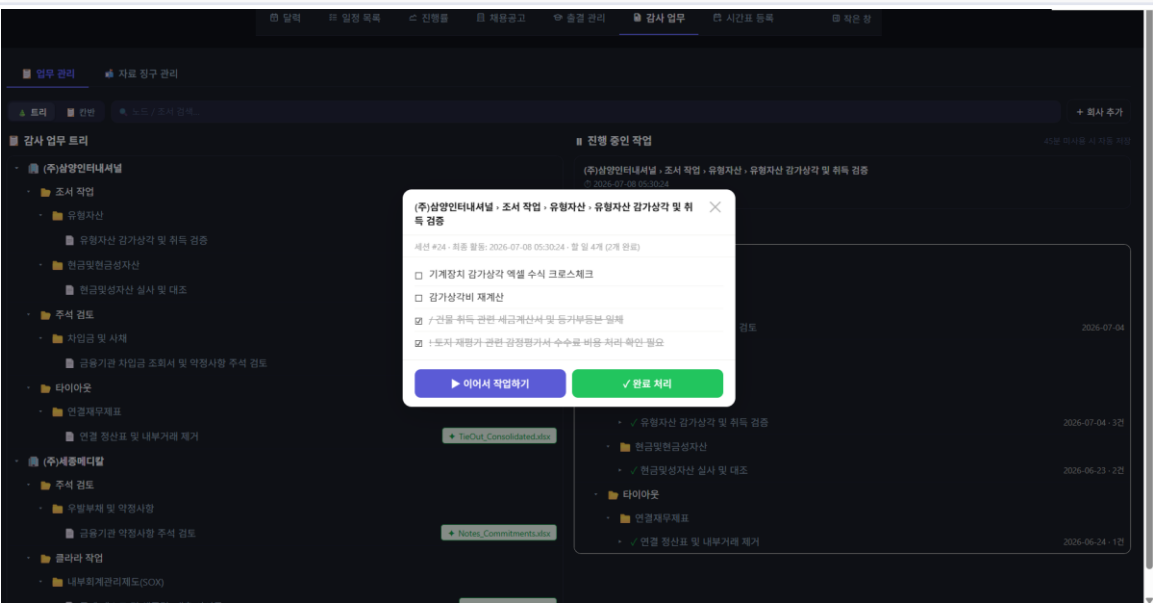


# 실행 화면 ② 칸반 뷰

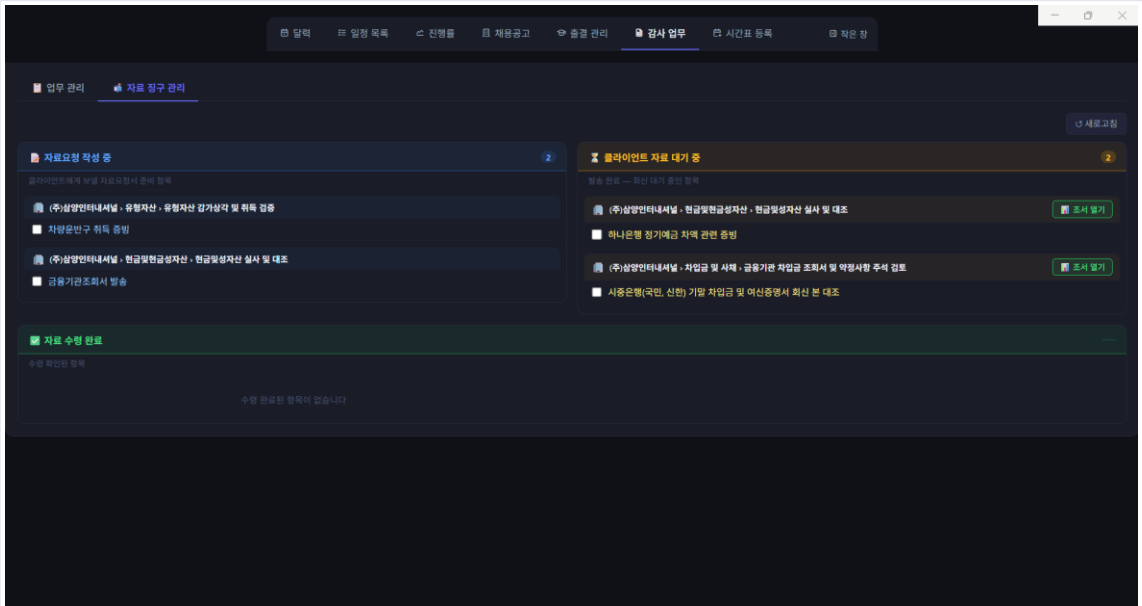


같은 데이터를 진행 상태별 칸반 보드로 전환해 확인

# 실행 화면 ③ 작업 상세 & 자료징구 현황

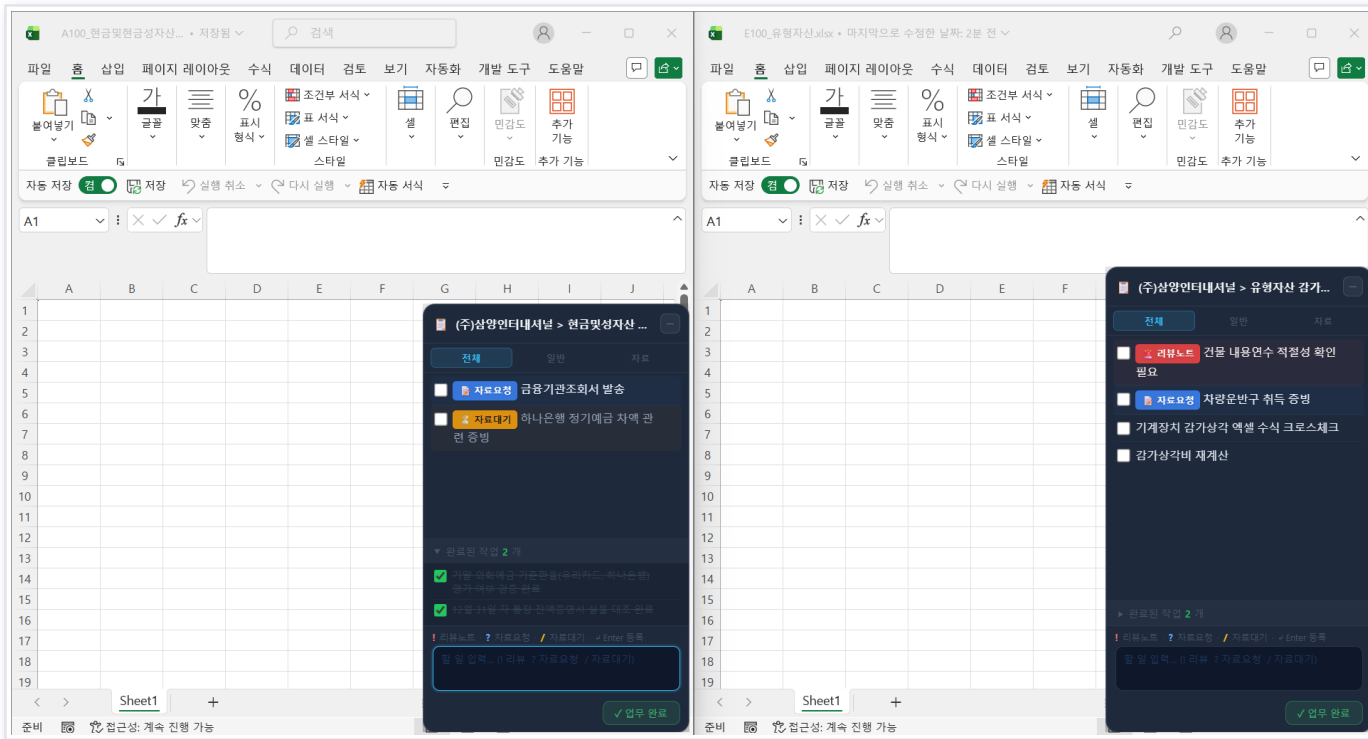


진행 중인 업무의 메모 · 할일 체크리스트 상세



감사 자료징구(요청 문서) 요청·수령 현황 트래킹

## 실행 화면 ④ 엑셀 자동 인식 & 자동저장



실제 작업 중인 워크페이퍼 옆에 뜬 미니 팝업 — 별도 조작 없이 자동 인식

### 화면에 보이지 않는 자동화 로직

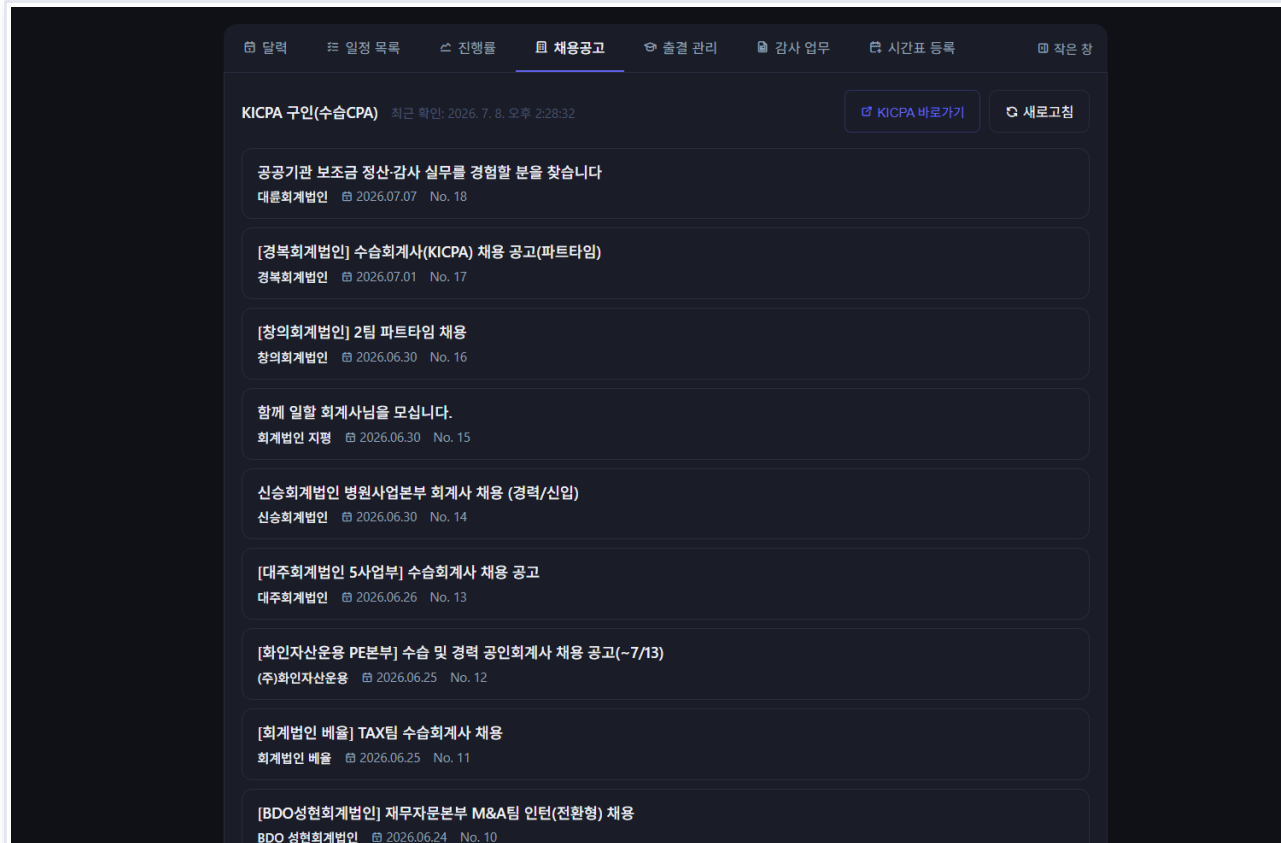
- 열려 있는 Excel 창 제목을 주기적으로 스캔해, 등록된 조서 파일명과 자동 매칭 후 담당 업무의 미니 팝업을 창 옆에 자동으로 띄움
- "[읽기 전용]", "- 복사본", ".xlsx" 등 파일명 장식을 제거하는 별도 파싱 로직으로 매칭 정확도 확보
- 45분간 입력이 없으면 해당 업무를 "진행 중인 작업"으로 자동 저장 (화면 우측 상단 문구)
- '\_전기', '\_py' 등 키워드가 포함된 파일은 감시 대상에서 자동 제외

## 어떻게 구현했나: Python ↔ Electron 창 추적



**예외 처리:** 최소화 시 좌표를 갱신하지 않아 복원 후 위치 틀어짐 방지 · 사용자가 팝업을 직접 옮기면 그 이후로는 자동 추적 대신 사용자 위치 존중 · Excel 종료 감지 시 세션 자동 정리

## 채용공고 자동수집 — 구현 방식



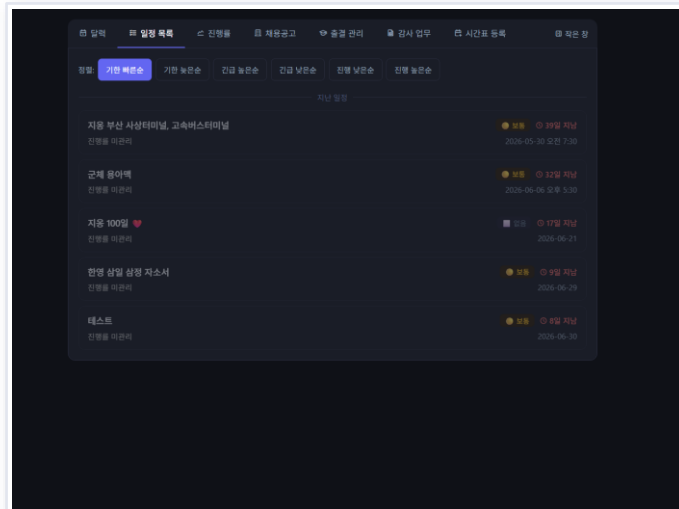
실제 수집된 KICPA 수습CPA 채용공고 목록 화면

### 어떻게 구현했나

- GitHub Actions가 매시 정각(하루 24회) KICPA 채용 게시판을 requests + BeautifulSoup으로 크롤링
- 목록 HTML의 onclick 속성에서 정규식으로 공고 ID(btnNo)를 추출, seenIds에 없는 신규 공고만 선별
- 스크래핑·저장은 매시 실행하되, 카카오톡 알림은 저녁 19~22시(신규 공고)·아침 6~9시(마감 전일)로 분리 발송 — 상태 파일로 중복 발송 방지
- 최초 실행 시 기존 공고 전체를 '확인 완료' 처리해, 과거 공고가 한꺼번에 알림으로 쏟아지는 것을 방지
- 크롤링 결과(kicpa\_jobs.json)는 GitHub Actions가 자동 커밋 → 앱은 다음 동기화 때 최신 공고 반영

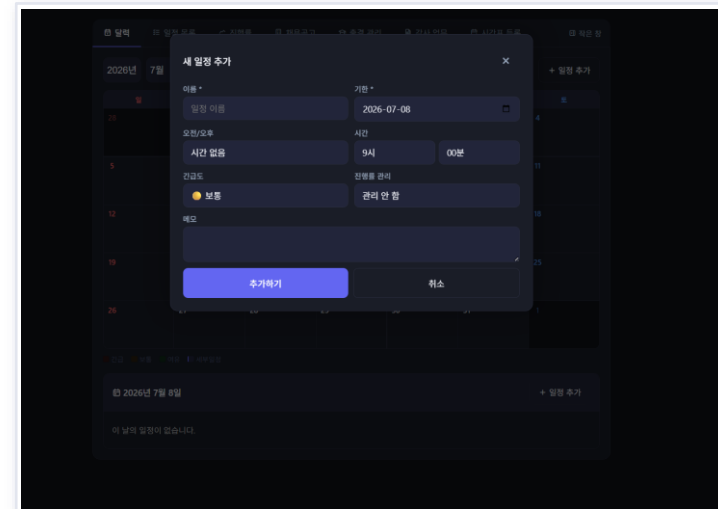
# 감사업무관리 · 채용공고 자동수집 외 기능

일정관리 앱은 위 기능들 외에도 아래 개인 일정관리 기능을 함께 제공합니다



일정 목록

마감일·진행률 기반 개인 일정 관리



일정 추가

긴급도 · 세부일정(무한 중첩) 지원

**그 외 백그라운드 자동화:** GitHub API 기반 데스크톱↔모바일 동기화 · 로컬 변경사항 자동 Git 백업(3분 주기)

## SUMMARY

# 이 프로젝트가 보여주는 것

### 일정관리 앱 — 기술 스택

- Electron + SQLite(better-sqlite3), 무한 계층 AuditNode 트리/칸반 관리
- Python(win32gui) ↔ Electron 간 프로세스 통신으로 Excel 창 실시간 추적
- GitHub Actions 기반 채용공고 크롤링(requests·BeautifulSoup) + 시간대 분리 카카오톡 알림
- GitHub API 동기화 · Windows 작업 스케줄러 기반 로컬→GitHub 자동 백업

---

감사 현장 업무의 흐름을 직접 관찰하고, 이를 트리·칸반 구조와 자동화 로직으로 구현해 본 경험입니다.